Lineage trace evidence pack

Item	Value
Reference	TRC-20260630-0057
Data point	EFR Credit Risk Summary · whole report
Reported value	Whole report
Reference date	2026-06-30
Result	Whole report (5 value columns) traced to 7 systems of record
Hops	28 (27 proven, 1 code-derived, 0 unresolved)
Generated by	tracer user (tracer)
Generated at	2026-10-16 09:11 UTC

TRC-20260630-0057
tracer user (tracer) · 2026-10-16 09:11 UTC

Hops

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
1	proven	fin_lake.raw.collateral -> fin_lake.stg.collateral	Copied from c.contract_id.	pipelines/spark/efr_job.py lines 54–56 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_collateral	Recalculated 899 agrees to stored value 899 (tolerance ±0.01) [collateral.row count]
2	proven	fin_lake.raw.counterparty -> fin_lake.stg.counterparty	Calculated as UPPER(d.sector_code); filtered to d.rn = 1.	pipelines/spark/efr_job.py lines 33–38 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_counterparty	Recalculated 400 agrees to stored value 400 (tolerance ±0.01) [counterparty.row count]
3	proven	fin_lake.raw.credit_limit -> fin_lake.stg.credit_limit	Calculated as CAST(l.limit_amount AS DECIMAL(20, 2)).	pipelines/spark/efr_job.py lines 58–61 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_credit_limit	Recalculated 247,252,793,500 agrees to stored value 247,252,793,500 (tolerance ±0.01) [credit_limit.limit_amount]
4	proven	fin_lake.raw.entity -> fin_lake.stg.entity	Calculated as UPPER(TRIM(e.entity_code)).	pipelines/spark/efr_job.py lines 29–31 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_entity	Recalculated 3 agrees to stored value 3 (tolerance ±0.01) [entity.row count]
5	code-derived	fin_lake.raw.fx_rate -> fin_lake.stg.fx_rate	Calculated as CAST(f.rate_to_usd AS DECIMAL(18, 8)); converted using FX rate f.rate_to_usd.	pipelines/spark/efr_job.py lines 63–65 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_fx_rate	Not reconciled: recalculated 12.33 vs stored 12.31 (tolerance ±0.01) [fx_rate.rate_to_usd]
6	proven	fin_lake.raw.loan_contract -> fin_lake.stg.loan	Calculated as CAST(l.undrawn_limit AS DECIMAL(20, 2)).	pipelines/spark/efr_job.py lines 46–52 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_loan	Recalculated 34,470,540,208.96 agrees to stored value 34,470,540,208.96 (tolerance ±0.01) [loan.undrawn_amt] (+1 more columns agree)
7	proven	fin_lake.raw.rating -> fin_lake.stg.rating	Copied from d.cpty_id; filtered to d.rn = 1.	pipelines/spark/efr_job.py lines 40–44 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.stg_rating	Recalculated 341 agrees to stored value 341 (tolerance ±0.01) [rating.row count]
8	proven	fin_lake.stg.counterparty -> fin_lake.core.dim_counterparty	Calculated as CASE c.sector_code WHEN 'SOV' THEN 'Sovereigns' WHEN 'BNK' THEN 'Banks' WHEN 'CORP' THEN 'Corporates' WHEN 'RET' THEN 'Retail' ELSE 'Other' END; mapped through a CASE with 4 branches.	pipelines/spark/efr_job.py lines 77–84 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_dim_counterparty	Recalculated 400 agrees to stored value 400 (tolerance ±0.01) [dim_counterparty.row count]
9	proven	fin_lake.stg.entity -> fin_lake.core.dim_entity	Copied from e.entity_code.	pipelines/spark/efr_job.py lines 72–75 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_dim_entity	Recalculated 3 agrees to stored value 3 (tolerance ±0.01) [dim_entity.row count]
10	proven	fin_lake.stg.loan -> fin_lake.core.dim_product	Calculated as CAST(CASE p.product_code WHEN 'RCF' THEN 0.50 WHEN 'TRADE' THEN 0.20 ELSE 1.00 END AS DECIMAL(9, 4)); mapped through a CASE with 2 branches.	pipelines/spark/efr_job.py lines 95–99 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_dim_product	Recalculated 2.7 agrees to stored value 2.7 (tolerance ±0.01) [dim_product.ccf] (+1 more columns agree)

TRC-20260630-0057 · tracer user (tracer) · 2026-10-16 09:11 UTC

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
11	proven	fin_lake.stg.rating -> fin_lake.core.dim_rating	Calculated as CAST(CASE WHEN g.rating_grade IN ('AAA', 'AA', 'A') THEN 0.35 WHEN g.rating_grade IN ('BBB', 'BB') THEN 0.45 ELSE 0.55 END AS DECIMAL(9, 6)); mapped through a CASE with 2 branches.	pipelines/spark/efr_job.py lines 86–93 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_dim_rating	Recalculated 3.6 agrees to stored value 3.6 (tolerance ±0.01) [dim_rating.lgd] (+1 more columns agree)
12	proven	fin_lake.core.dim_counterparty -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 278,353 agrees to stored value 278,353 (tolerance ±0.01) [fact_exposure.cpty_key]
13	proven	fin_lake.core.dim_entity -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 2,804 agrees to stored value 2,804 (tolerance ±0.01) [fact_exposure.entity_key]
14	proven	fin_lake.core.dim_product -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 3,538 agrees to stored value 3,538 (tolerance ±0.01) [fact_exposure.product_key] (+2 more columns agree)
15	proven	fin_lake.core.dim_rating -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 6,748 agrees to stored value 6,748 (tolerance ±0.01) [fact_exposure.rating_key] (+2 more columns agree)
16	proven	fin_lake.stg.collateral -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 5,822,029,359.1 agrees to stored value 5,822,029,359.1 (tolerance ±0.01) [fact_exposure.ead_usd] (+2 more columns agree)
17	proven	fin_lake.stg.fx_rate -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 5,822,029,359.1 agrees to stored value 5,822,029,359.1 (tolerance ±0.01) [fact_exposure.ead_usd] (+2 more columns agree)
18	proven	fin_lake.stg.loan -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 5,822,029,359.1 agrees to stored value 5,822,029,359.1 (tolerance ±0.01) [fact_exposure.ead_usd] (+2 more columns agree)
19	proven	fin_lake.stg.rating -> fin_lake.core.fact_exposure	Calculated as ROUND(b.drawn_usd + b.ccf * b.undrawn_usd, 2).	pipelines/spark/efr_job.py lines 101–131 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_exposure	Recalculated 5,822,029,359.1 agrees to stored value 5,822,029,359.1 (tolerance ±0.01) [fact_exposure.ead_usd] (+2 more columns agree)
20	proven	fin_lake.core.dim_entity -> fin_lake.core.fact_limit	Calculated as ROUND(l.limit_amount * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/spark/efr_job.py lines 133–140 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_limit	Recalculated 24 agrees to stored value 24 (tolerance ±0.01) [fact_limit.entity_key]
21	proven	fin_lake.stg.credit_limit -> fin_lake.core.fact_limit	Calculated as ROUND(l.limit_amount * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/spark/efr_job.py lines 133–140 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_limit	Recalculated 8,063,137,244.32 agrees to stored value 8,063,137,244.32 (tolerance ±0.01) [fact_limit.limit_usd]

TRC-20260630-0057 · tracer user (tracer) · 2026-10-16 09:11 UTC

Hop	Status	From -> To	Transformation	Evidence	Reconciliation
22	proven	fin_lake.stg.fx_rate -> fin_lake.core.fact_limit	Calculated as ROUND(l.limit_amount * fx.rate_to_usd, 2); converted using FX rate fx.rate_to_usd.	pipelines/spark/efr_job.py lines 133–140 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.core_fact_limit	Recalculated 8,063,137,244.32 agrees to stored value 8,063,137,244.32 (tolerance ±0.01) [fact_limit.limit_usd]
23	proven	fin_lake.core.dim_counterparty -> fin_lake.rpt.efr_credit_risk_summary	Calculated as ROUND(x.ead_usd / 1000000, 2).	pipelines/spark/efr_job.py lines 143–159 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.rpt_efr_credit_risk_summary	Recalculated 5,822.03 agrees to stored value 5,822.03 (tolerance ±0.01) [efr_credit_risk_summary.exposure_ead] (+3 more columns agree)
24	proven	fin_lake.core.dim_entity -> fin_lake.rpt.efr_credit_risk_summary	Calculated as ROUND(x.ead_usd / 1000000, 2).	pipelines/spark/efr_job.py lines 143–159 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.rpt_efr_credit_risk_summary	Recalculated 5,822.03 agrees to stored value 5,822.03 (tolerance ±0.01) [efr_credit_risk_summary.exposure_ead] (+3 more columns agree)
25	proven	fin_lake.core.fact_exposure -> fin_lake.rpt.efr_credit_risk_summary	Calculated as ROUND(x.ead_usd / 1000000, 2).	pipelines/spark/efr_job.py lines 143–159 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.rpt_efr_credit_risk_summary	Recalculated 5,822.03 agrees to stored value 5,822.03 (tolerance ±0.01) [efr_credit_risk_summary.exposure_ead] (+2 more columns agree)
26	proven	fin_lake.core.fact_limit -> fin_lake.rpt.efr_credit_risk_summary	Calculated as ROUND(x.ead_usd / 1000000, 2).	pipelines/spark/efr_job.py lines 143–159 · OpenLineage job efr_credit_risk.execute_save_into_data_source_command.rpt_efr_credit_risk_summary	Recalculated 8,063.14 agrees to stored value 8,063.14 (tolerance ±0.01) [efr_credit_risk_summary.limit_amount]
27	proven	fin_lake.rpt.efr_credit_risk_summary -> fin_lake.rpt.v_efr_credit_risk_summary	Copied from exposure_ead.	fin_lake: view rpt.v_efr_credit_risk_summary · definition sha256 bb2aeceb14f3… · captured 2026-10-16	Recalculated 11,644.06 agrees to stored value 11,644.06 (tolerance ±0.01) [v_efr_credit_risk_summary.exposure_ead] (+4 more columns agree)
28	proven	fin_lake.rpt.v_efr_credit_risk_summary -> superset.EFR Credit Risk Summary	Superset dataset 'efr_credit_risk_summary' is a virtual dataset over SQL; the chart displays it as a table.	Superset chart 23 'EFR Credit Risk Summary' -> dataset efr_credit_risk_summary	Superset chart data re-run returned all 13 reported rows; column totals of rpt.v_efr_credit_risk_summary agree with the report

TRC-20260630-0057 tracer user (tracer) · 2026-10-16 09:11 UTC

SQL excerpts

Hop 1: ("stg.collateral", """ SELECT c.collateral_id, c.contract_id, c.collateral_type, c.ccy, CAST(c.market_value AS DECIMAL(20,2)) AS market_value FROM raw_collateral c"""),

Hop 2: ("stg.counterparty", """ -- dedupe: keep the latest delivery per counterparty; flag intragroup SELECT d.cpty_id, TRIM(d.cpty_name) AS cpty_name, d.country, UPPER(d.sector_code) AS sector_code, (d.is_intragroup = 'Y') AS is_intragroup FROM (SELECT c.*, ROW_NUMBER() OVER (PARTITION BY c.cpty_id ORDER BY c._loaded_at DESC) AS rn FROM raw_counterparty c) d WHERE d.rn = 1"""),

Hop 3: ("stg.credit_limit", """ SELECT UPPER(l.entity_code) AS entity_code, UPPER(l.sector_code) AS sector_code, CAST(l.limit_amount AS DECIMAL(20,2)) AS limit_amount, l.ccy FROM raw_credit_limit l"""),

Hop 4: ("stg.entity", """ SELECT UPPER(TRIM(e.entity_code)) AS entity_code, TRIM(e.entity_name) AS entity_name, e.country, e.consolidation_group FROM raw_entity e"""),

Hop 5: ("stg.fx_rate", """ SELECT f.ccy, CAST(f.rate_date AS DATE) AS rate_date, CAST(f.rate_to_usd AS DECIMAL(18,8)) AS rate_to_usd FROM raw_fx_rate f"""),

Hop 6: ("stg.loan", """ -- drawn amount = principal + accrued interest - impairment (transaction currency) SELECT l.contract_id, UPPER(l.entity_code) AS entity_code, l.cpty_id, UPPER(l.product_code) AS product_code, l.ccy, CAST(l.principal AS DECIMAL(20,2)) + CAST(l.accrued_int AS DECIMAL(20,2)) - CAST(l.impairment AS DECIMAL(20,2)) AS drawn_amt, CAST(l.undrawn_limit AS DECIMAL(20,2)) AS undrawn_amt, CAST(l.start_date AS DATE) AS start_date, CAST(l.maturity_date AS DATE) AS maturity_date FROM raw_loan_contract l"""),

Hop 7: ("stg.rating", """ -- latest rating per counterparty SELECT d.cpty_id, UPPER(d.rating_grade) AS rating_grade, CAST(d.rating_date AS DATE) AS rating_date, d.agency FROM (SELECT r.*, ROW_NUMBER() OVER (PARTITION BY r.cpty_id ORDER BY r.rating_date DESC) AS rn FROM raw_rating r) d WHERE d.rn = 1"""),

Hop 8: ("core.dim_counterparty", """ -- portfolio derived from the sector code SELECT CAST(ROW_NUMBER() OVER (ORDER BY c.cpty_id) AS INT) AS cpty_key, c.cpty_id, c.cpty_name, c.country, c.sector_code, CASE c.sector_code WHEN 'SOV' THEN 'Sovereigns' WHEN 'BNK' THEN 'Banks' WHEN 'CORP' THEN 'Corporates' WHEN 'RET' THEN 'Retail' ELSE 'Other' END AS portfolio, c.is_intragroup FROM stg_counterparty c"""),

Hop 9: ("core.dim_entity", """ SELECT CAST(ROW_NUMBER() OVER (ORDER BY e.entity_code) AS INT) AS entity_key, e.entity_code, e.entity_name, e.country, e.consolidation_group FROM stg_entity e"""),

Hop 10: ("core.dim_product", """ SELECT CAST(ROW_NUMBER() OVER (ORDER BY p.product_code) AS INT) AS product_key, p.product_code, CAST(CASE p.product_code WHEN 'RCF' THEN 0.50 WHEN 'TRADE' THEN 0.20 ELSE 1.00 END AS DECIMAL(9,4)) AS ccf, CAST(CASE p.product_code WHEN 'GUAR' THEN 1.00 WHEN 'TRADE' THEN 0.50 WHEN 'RCF' THEN 0.80 ELSE 0.90 END AS DECIMAL(9,4)) AS risk_weight_base FROM (SELECT DISTINCT l.product_code FROM stg_loan l) p"""),

Hop 11: ("core.dim_rating", """ -- grade -> probability of default and loss-given-default band ('NR' = not rated) SELECT CAST(ROW_NUMBER() OVER (ORDER BY g.rating_grade) AS INT) AS rating_key, g.rating_grade, CAST(CASE g.rating_grade WHEN 'AAA' THEN 0.0003 WHEN 'AA' THEN 0.0006 WHEN 'A' THEN 0.0015 WHEN 'BBB' THEN 0.0050 WHEN 'BB' THEN 0.0200 WHEN 'B' THEN 0.0600 WHEN 'CCC' THEN 0.2000 ELSE 0.0300 END AS DECIMAL(9,6)) AS pd, CAST(CASE WHEN g.rating_grade IN ('AAA', 'AA', 'A') THEN 0.35 WHEN g.rating_grade IN ('BBB', 'BB') THEN 0.45 ELSE 0.55 END AS DECIMAL(9,6)) AS lgd FROM (SELECT r.rating_grade FROM stg_rating r UNION SELECT 'NR' AS rating_grade) g"""),

Hop 12: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 13: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 14: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 15: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 16: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 17: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 18: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 19: ("core.fact_exposure", """ -- EAD = drawn + CCF x undrawn (USD at the 30-Jun-2026 rate); RWA = EAD x risk weight; EL = PD x LGD x EAD; intragroup excluded WITH base AS (SELECT e.entity_key, c.cpty_key, p.product_key, dr.rating_key, l.contract_id, ROUND(l.drawn_amt * fx.rate_to_usd, 2) AS drawn_usd, ROUND(l.undrawn_amt * fx.rate_to_usd, 2) AS undrawn_usd, p.ccf, p.risk_weight_base AS risk_weight, dr.pd, dr.lgd FROM stg_loan l JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30' JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN core_dim_counterparty c ON c.cpty_id = l.cpty_id JOIN core_dim_product p ON p.product_code = l.product_code LEFT JOIN stg_rating r ON r.cpty_id = l.cpty_id JOIN core_dim_rating dr ON dr.rating_grade = COALESCE(r.rating_grade, 'NR') WHERE c.is_intragroup = FALSE), coll AS (SELECT k.contract_id, SUM(ROUND(k.market_value * fx.rate_to_usd, 2)) AS collateral_usd FROM stg_collateral k JOIN stg_fx_rate fx ON fx.ccy = k.ccy AND fx.rate_date = DATE '2026-06-30' GROUP BY k.contract_id) SELECT DATE '2026-06-30' AS as_o

Hop 20: ("core.fact_limit", """ SELECT DATE '2026-06-30' AS as_of, e.entity_key, CASE l.sector_code WHEN 'SOV' THEN 'Sovereigns' WHEN 'BNK' THEN 'Banks' WHEN 'CORP' THEN 'Corporates' WHEN 'RET' THEN 'Retail' ELSE 'Other' END AS portfolio, ROUND(l.limit_amount * fx.rate_to_usd, 2) AS limit_usd FROM stg_credit_limit l JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30'"""),

Hop 21: ("core.fact_limit", """ SELECT DATE '2026-06-30' AS as_of, e.entity_key, CASE l.sector_code WHEN 'SOV' THEN 'Sovereigns' WHEN 'BNK' THEN 'Banks' WHEN 'CORP' THEN 'Corporates' WHEN 'RET' THEN 'Retail' ELSE 'Other' END AS portfolio, ROUND(l.limit_amount * fx.rate_to_usd, 2) AS limit_usd FROM stg_credit_limit l JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30'"""),

Hop 22: ("core.fact_limit", """ SELECT DATE '2026-06-30' AS as_of, e.entity_key, CASE l.sector_code WHEN 'SOV' THEN 'Sovereigns' WHEN 'BNK' THEN 'Banks' WHEN 'CORP' THEN 'Corporates' WHEN 'RET' THEN 'Retail' ELSE 'Other' END AS portfolio, ROUND(l.limit_amount * fx.rate_to_usd, 2) AS limit_usd FROM stg_credit_limit l JOIN core_dim_entity e ON e.entity_code = l.entity_code JOIN stg_fx_rate fx ON fx.ccy = l.ccy AND fx.rate_date = DATE '2026-06-30'"""),

Hop 23: ("rpt.efr_credit_risk_summary", """ -- USD millions, 2 dp WITH expo AS (SELECT f.entity_key, c.portfolio, SUM(f.ead_usd) AS ead_usd, SUM(f.rwa_usd) AS rwa_usd, SUM(f.el_usd) AS el_usd FROM core_fact_exposure f JOIN core_dim_counterparty c ON c.cpty_key = f.cpty_key GROUP BY f.entity_key, c.portfolio), lim AS (SELECT l.entity_key, l.portfolio, SUM(l.limit_usd) AS limit_usd FROM core_fact_limit l GROUP BY l.entity_key, l.portfolio) SELECT e.entity_code AS entity, 'CREDIT' AS risk_type, x.portfolio, ROUND(x.ead_usd / 1000000, 2) AS exposure_ead, ROUND(x.rwa_usd / 1000000, 2) AS rwa, ROUND(x.el_usd / 1000000, 2) AS expected_loss, ROUND(l.limit_usd / 1000000, 2) AS limit_amount, DATE '2026-06-30' AS as_of FROM expo x JOIN core_dim_entity e ON e.entity_key = x.entity_key LEFT JOIN lim l ON l.entity_key = x.entity_key AND l.portfolio = x.portfolio"""),

Hop 24: ("rpt.efr_credit_risk_summary", """ -- USD millions, 2 dp WITH expo AS (SELECT f.entity_key, c.portfolio, SUM(f.ead_usd) AS ead_usd, SUM(f.rwa_usd) AS rwa_usd, SUM(f.el_usd) AS el_usd FROM core_fact_exposure f JOIN core_dim_counterparty c ON c.cpty_key = f.cpty_key GROUP BY f.entity_key, c.portfolio), lim AS (SELECT l.entity_key, l.portfolio, SUM(l.limit_usd) AS limit_usd FROM core_fact_limit l GROUP BY l.entity_key, l.portfolio) SELECT e.entity_code AS entity, 'CREDIT' AS risk_type, x.portfolio, ROUND(x.ead_usd / 1000000, 2) AS exposure_ead, ROUND(x.rwa_usd / 1000000, 2) AS rwa, ROUND(x.el_usd / 1000000, 2) AS expected_loss, ROUND(l.limit_usd / 1000000, 2) AS limit_amount, DATE '2026-06-30' AS as_of FROM expo x JOIN core_dim_entity e ON e.entity_key = x.entity_key LEFT JOIN lim l ON l.entity_key = x.entity_key AND l.portfolio = x.portfolio"""),

Hop 25: ("rpt.efr_credit_risk_summary", """ -- USD millions, 2 dp WITH expo AS (SELECT f.entity_key, c.portfolio, SUM(f.ead_usd) AS ead_usd, SUM(f.rwa_usd) AS rwa_usd, SUM(f.el_usd) AS el_usd FROM core_fact_exposure f JOIN core_dim_counterparty c ON c.cpty_key = f.cpty_key GROUP BY f.entity_key, c.portfolio), lim AS (SELECT l.entity_key, l.portfolio, SUM(l.limit_usd) AS limit_usd FROM core_fact_limit l GROUP BY l.entity_key, l.portfolio) SELECT e.entity_code AS entity, 'CREDIT' AS risk_type, x.portfolio, ROUND(x.ead_usd / 1000000, 2) AS exposure_ead, ROUND(x.rwa_usd / 1000000, 2) AS rwa, ROUND(x.el_usd / 1000000, 2) AS expected_loss, ROUND(l.limit_usd / 1000000, 2) AS limit_amount, DATE '2026-06-30' AS as_of FROM expo x JOIN core_dim_entity e ON e.entity_key = x.entity_key LEFT JOIN lim l ON l.entity_key = x.entity_key AND l.portfolio = x.portfolio"""),

Hop 26: ("rpt.efr_credit_risk_summary", """ -- USD millions, 2 dp WITH expo AS (SELECT f.entity_key, c.portfolio, SUM(f.ead_usd) AS ead_usd, SUM(f.rwa_usd) AS rwa_usd, SUM(f.el_usd) AS el_usd FROM core_fact_exposure f JOIN core_dim_counterparty c ON c.cpty_key = f.cpty_key GROUP BY f.entity_key, c.portfolio), lim AS (SELECT l.entity_key, l.portfolio, SUM(l.limit_usd) AS limit_usd FROM core_fact_limit l GROUP BY l.entity_key, l.portfolio) SELECT e.entity_code AS entity, 'CREDIT' AS risk_type, x.portfolio, ROUND(x.ead_usd / 1000000, 2) AS exposure_ead, ROUND(x.rwa_usd / 1000000, 2) AS rwa, ROUND(x.el_usd / 1000000, 2) AS expected_loss, ROUND(l.limit_usd / 1000000, 2) AS limit_amount, DATE '2026-06-30' AS as_of FROM expo x JOIN core_dim_entity e ON e.entity_key = x.entity_key LEFT JOIN lim l ON l.entity_key = x.entity_key AND l.portfolio = x.portfolio"""),

Hop 27: CREATE VIEW "rpt"."v_efr_credit_risk_summary" AS SELECT entity, risk_type, portfolio, exposure_ead, rwa, expected_loss, limit_amount, utilisation_pct FROM (SELECT s.entity, s.risk_type, s.portfolio, s.exposure_ead, s.rwa, s.expected_loss, s.limit_amount, round(((s.exposure_ead / NULLIF(s.limit_amount, (0)::numeric)) * (100)::numeric), 2) AS utilisation_pct FROM rpt.efr_credit_risk_summary s UNION ALL SELECT 'GROUP'::text AS entity, 'CREDIT'::text AS risk_type, 'ALL'::text AS portfolio, sum(s.exposure_ead) AS sum, sum(s.rwa) AS sum, sum(s.expected_loss) AS sum, sum(s.limit_amount) AS sum, round(((sum(s.exposure_ead) / NULLIF(sum(s.limit_amount), (0)::numeric)) * (100)::numeric), 2) AS utilisation_pct FROM rpt.efr_credit_risk_summary s) u ORDER BY (entity = 'GROUP'::text), entity, portfolio

Hop 28: SELECT entity, risk_type, portfolio, exposure_ead, rwa, expected_loss, limit_amount, utilisation_pct FROM rpt.v_efr_credit_risk_summary

TRC-20260630-0057 · tracer user (tracer) · 2026-10-16 09:11 UTC